Part2

Problem 1:

```
                          πSid,Cid
                              |
                              ⋈
                            /    \
                          ⋈          πCid,Ctitle,Instructor
                        /      \                   |
   πSid,Sname,Major        Enroll          σCredits=3
           |                                     |
    σGpa>3.4                                Course
           |
      Student
```

Problem 2:

Step1:

Rule 1: Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

Rule used: Conjunctive selection decomposition (Rule 1)

$\sigma\{\theta1 \wedge \theta2\}(E) = \sigma\theta1(\sigma\theta2(E))$

Then $\sigma\{\theta1 \wedge \theta2\}(E1 \bowtie \{\theta3\}E2) = \sigma\theta1(\sigma\theta2(E1 \bowtie \{\theta3\}E2))$

Step2:

Rule 7 (a): When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

Rule used: Selection pushdown through join (Rule 7(a))

$\sigma\theta(E1 \bowtie \{\theta3\}E2) = E1 \bowtie \{\theta3\}(\sigma\theta(E2))$ when θ involves only attributes of E2

Since $\theta2$ involves only attributes of E2

Then $\sigma\theta1(\sigma\theta2(E1 \bowtie \{\theta3\}E2)) = \sigma\theta1(E1 \bowtie \{\theta3\}(\sigma\theta2(E2)))$

Step3:

$\sigma\{\theta1 \wedge \theta2\}(E1 \bowtie \{\theta3\}E2) = \sigma\theta1(E1 \bowtie \{\theta3\}(\sigma\theta2(E2)))$

Problem3:

Selectivity(Major='Math') = 1/100

n(σ{Major='Math'}(Student)) = 20000 * (1/100) = 200

Selectivity(Gpa＞3.00) = (4.00-3.00)/(4.00-0.00) = 0.25

n(σ{Gpa＞3.00}(Student)) = 20000 * 0.25 = 5000

(a)

Selectivity(Major='Math')∧(Gpa>3.00)}(Student)) = (1/100) * 0.25 = 0.0025

n(σ{(Major='Math')∧(Gpa>3.00)}(Student)) = 20000 * 0.0025 = 50

(b)

Selectivity(θ1∨θ2) = Sc(θ1) + Sc(θ2) − Sc(θ1∧θ2)

Selectivity((Major='Math') ∨ (Gpa>3.00)) = Selectivity(Major='Math') + Selectivity(Gpa>3.00) − Selectivity((Major='Math')∧(Gpa>3.00)) = 0.01+0.25−0.0025 = 0.2575

n(σ{(Major='Math')∨(Gpa>3.00)}(Student)) = 20000 * 0.2575 = 5150

(c)

Selectivity(Major≠'Math') = 1 - 1/100 = 99/100

Selectivity((Major≠'Math')∧(Gpa>3.00)) = (99/100) * 0.25 = 0.2475

n(σ{(Major≠'Math')∧(Gpa>3.00)}(Student)) = 20000 * 0.2475 = 4950

Problem4:

Step 1: Push selection down

$$\pi_{Sid,Dean}\ \sigma_{Major='Math'}(Student \bowtie_{Major=Dcode}(Department \bowtie_{School=Scode} School)) =$$
$$\pi_{Sid,Dean}((\sigma_{Major='Math'}(Student)) \bowtie_{Major=Dcode}(Department \bowtie_{School=Scode} School))$$

Step 2: Push projections down

Optimized Query 1:
$$\pi_{Sid,Dean}(\pi_{Sid,Major}(\sigma_{Major='Math'}(Student)) \bowtie_{Major=Dcode}(\pi_{Dcode,School}(Department) \bowtie_{School=Scode} \pi_{Scode,Dean}(School))$$

Optimized Query 2:
$$\pi_{Sid,Dean}((\pi_{Sid,Major}(\sigma_{Major='Math'}(Student)) \bowtie_{Major=Dcode} \pi_{Dcode,School}(Department)) \bowtie_{School=Scode} \pi_{Scode,Dean}(School)$$

Problem5:

1.Relation sizes and number of blocks

Student:
Tuple size:
10+30+10+5 = 55 bytes
Blocking factor:
bfr(Student) = [500/55] = 9
Number of blocks:
B(Student) = [20000/9] = 2223

Department:
Tuple size:
10+30+10+50+10 = 110 bytes
Blocking factor:
bfr(Department) = [500/110] = 4
Number of blocks:
B(Department) = [200/4] = 50

School:
Tuple size:
10+30+30+50+100=220 bytes
Blocking factor:
bfr(School) = [500/220] = 2
Number of blocks:
B(School) = [20/2] = 10

2. Intermediate result sizes

Because of the foreign-key constraints:
①each Student matches exactly one Department
②each Department matches exactly one School
So:
Student⋈Department:
Number of tuples:
T(Student⋈Department) = 20000
Tuple size (duplicate join column kept):
55+110 = 165 bytes
Blocking factor:
bfr(SD) = [500/165] = 3
Blocks: B(SD) = [20000/3] = 6667

Department⋈School:

Number of tuples:

T(Department⋈School) = 200

Tuple size:

110+220 = 330 bytes

Blocking factor:

bfr(DS) = [500/330] = 1

Blocks: B(DS) = 200

Final result:

Either way, final join result has:

T(final) = 20000

Tuple size:

55+110+220 = 385 bytes

Blocking factor:

bfr(final) = [500/385] = 1

Blocks:

B(final)=20000

3. Three evaluation plans

Plan1:

```
                    ⋈ (School=Scode)Index NL Join
 Index NL Join    /                        \
        ⋈ (Major=Dcode)                School
          /              \
   Student         Department
```

Plan2:

```
                    ⋈ (Major=Dcode)Block NL Join
                   /                        \
         Student                ⋈ (School=Scode)Index NL Join
                                 /                 \
                     Department           School
```

Plan3:

```
                    ⋈ (School=Scode)Index NL Join
                   /                               \
      Block NL Join⋈ (Major=Dcode)                School
          /              \
   Student         Department
```

Plan 1

First do Student⋈Department using index nested-loop join, then join with School using index

nested-loop join.
(Student⋈{Major=Dcode}Department)⋈ {School=Scode}School

Step 1:
scan outer Student once: B(Student) = 2223
for each Student tuple, probe Department index:
index height = 4
then 1 more block to fetch matching Department tuple
So per Student tuple:
4+1 = 5
Total index lookup cost:
20000×5=100000
write intermediate SD: B(SD)=6667
So step 1 cost:
2223+100000+6667=108890

Step 2:
SD⋈School
Use SD as outer, and use the primary index on School.Scode as inner.
Cost:
scan outer SD: B(SD) = 6667
for each SD tuple, probe School index:
index height = 2
plus 1 data block
So per tuple:
2+1=3
Total lookup cost:
20000×3 = 60000
write final result:
B(final) = 20000
So step 2 cost:
6667+60000+20000 = 86667
Total cost of Plan 1
108890+86667 = 195557

Plan 2:
First do Department⋈School using index nested-loop join, then join the result with Student using block nested-loop join.
Student⋈{Major=Dcode}(Department⋈{School=Scode}School)
To reduce cost in the second join, use the smaller relation DS as the outer.

Step 1:
Department⋈School
Use Department as outer, School as indexed inner.

Cost:
scan Department once:
B(Department) = 50
for each Department tuple, probe School index:
height 2
plus 1 data block
Per tuple:
2+1 = 3
Total lookup cost: 200×3 = 600
write intermediate DS:
B(DS) = 200
So step 1 cost:
50+600+200 = 850

Step 2:
Student⋈DS
Use block nested-loop join, with DS as outer and Student as inner.
Cost formula:
B(outer)+B(outer)*B(inner)+B(output)
So:
200+200*2223+20000 = 200+444600+20000 = 464800
Total cost of Plan 2
850+464800 = 465650

Plan 3:
First do Student⋈Department using block nested-loop join, then join the result with School using index nested-loop join.
(Student⋈{Major=Dcode}Department)⋈ {School=Scode}School
For the first join, use the smaller relation Department as outer.

Step 1:
Department⋈Student
Use Department as outer, Student as inner in block nested-loop join.
Cost: B(Department)+B(Department)*B(Student)+B(SD) = 50+50*2223+6667 = 50+111150+6667 = 117867

Step 2:
SD⋈School
Use index nested-loop join with SD as outer and School as indexed inner.
As computed before:
6667+20000*3+20000 = 6667+60000+20000 = 86667
Total cost of Plan 3
117867+86667 = 204534

Among these three plans: Plan 1 is the cheapest. because both joins can exploit the available primary indexes on: Department.Dcode and School.Scode.